

Swagger Petstore API Automation Assessment

Git Repository:

[“ust-sdet-training/SDET: Repository for SDET training batch”](#)

The complete source code, framework implementation, test cases, and supporting files for this assessment are available in the above Git repository.

1. Project Overview

This project automates the Swagger Petstore APIs using Rest Assured and JUnit 5.

The framework covers:

- GET Requests
- POST Requests
- PUT Requests
- DELETE Requests
- Query Parameters
- Request Payload Validation
- Response Validation
- Dynamic Data Handling
- Assertions
- API Chaining
- Test Organization

Base URL:

<https://petstore.swagger.io/v2>

2. Technology Stack

Technology Purpose

Java Programming Language

Rest Assured API Automation

JUnit 5 Test Execution & Assertions

Maven Build Tool

Jackson JSON Serialization / Deserialization

3. Project Structure

src

└─ test

│ └─ java

│

│ └─ model

│ │ └─ PetResponse.java

│ │ └─ OrderResponse.java

│ │ └─ UserResponse.java

│

│ └─ tests

│ │ └─ PetApiTest.java

│ │ └─ StoreApiTest.java

│ │ └─ UserApiTest.java

│

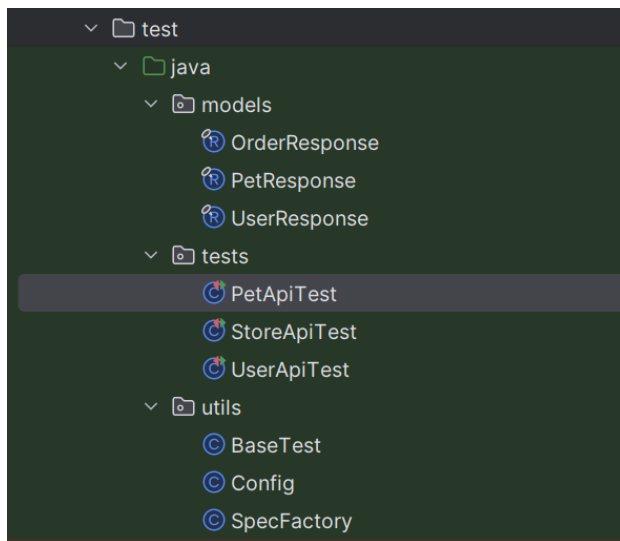
└─ utils

└─ BaseTest.java

└─ Config.java

└─ SpecFactory.java

Screenshot



4. Framework Design

BaseTest

Responsible for:

- Initial framework setup
- Base URI configuration

SpecFactory

Responsible for:

- Request Specifications
- Response Specifications
- Endpoint Management
- Common Reusable Configurations

Model Classes

Implemented using Java Records.

Purpose:

- Deserialize API Responses
- Perform Assertions

Test Classes

Contain API test scenarios for:

- Pet APIs
 - Store APIs
 - User APIs
-

5. API Coverage

Pet Module

Covered Endpoints:

POST /pet

GET /pet/{petId}

PUT /pet

DELETE /pet/{petId}

GET /pet/findByStatus

Validations:

- Status Code Validation
 - Response Body Validation
 - Dynamic ID Extraction
 - Query Parameter Validation
-

Store Module

Covered Endpoints:

POST /store/order

GET /store/order/{orderId}

Validations:

- Order Creation
 - Order Retrieval
 - Response Assertions
-

User Module

Covered Endpoints:

POST /user

GET /user/{username}

PUT /user/{username}

GET /user/login

Validations:

- User Creation
 - User Retrieval
 - User Update
 - Login Verification
-

6. Request Payload Strategy

Request payloads are created using:

Map<String,Object>

Example:

```
Map<String,Object> petBody = Map.of(
    "id",1001,
    "name","Tommy",
    "status","available"
);
```

Reason:

- Lightweight
 - Easy to Maintain
 - Suitable for API Payload Construction
-

7. Response Handling Strategy

Response models are implemented using Java Records.

Example:

```
@JsonIgnoreProperties(ignoreUnknown = true)
```

```
public record PetResponse(
```

```
    Long id,
```

```
    String name,
```

```
    String status
```

```
){
```

```
}
```

Reason:

- Immutable
- Clean
- Minimal Code
- Suitable for Assertions

8. Assertions Implemented

The following assertions are used:

Status Code Validation

```
assertEquals(200, response.statusCode());
```

Response Body Validation

```
assertEquals("Tommy", pet.name());
```

Dynamic Value Validation

```
assertEquals(
```

```
    petId,
```

```
    fetchedPet.id()
```

```
);
```

Collection Validation

```
assertTrue(pets.length > 0);
```

9. Reusable Specifications

Request Specification

Contains:

- Content Type
- Common Headers

Response Specification

Contains:

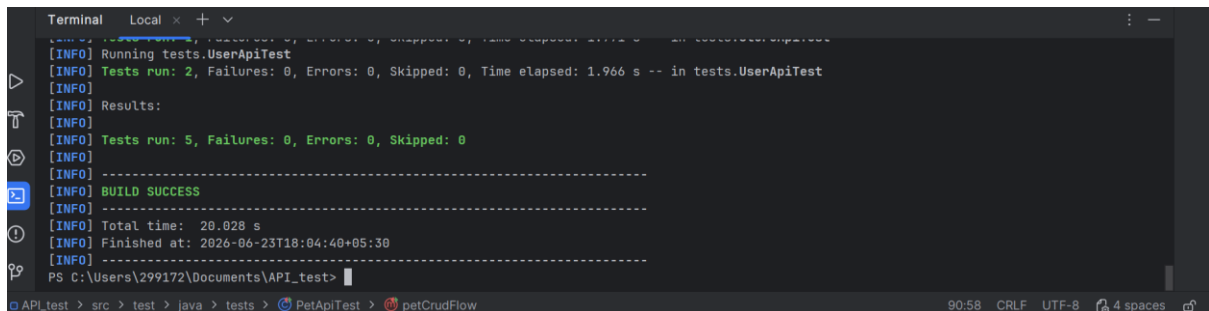
- Expected Status Code
 - Content Type Validation
 - Response Time Validation
-

10. Test Execution

Command Used:

mvn clean test

Screenshot



```
Terminal Local x + v
[INFO] Running tests.UserApiTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.966 s -- in tests.UserApiTest
[INFO] Results:
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time: 20.028 s
[INFO] Finished at: 2024-06-23T18:04:40+05:30
[INFO]
PS C:\Users\299172\Documents\API_test>
```

The screenshot shows a terminal window with the following output: [INFO] Running tests.UserApiTest, [INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.966 s -- in tests.UserApiTest, [INFO] Results:, [INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, [INFO] BUILD SUCCESS, [INFO] Total time: 20.028 s, [INFO] Finished at: 2024-06-23T18:04:40+05:30, [INFO]. The terminal prompt is PS C:\Users\299172\Documents\API_test>.

11. Test Results

Total Test Classes:

- PetApiTest
- StoreApiTest
- UserApiTest

Executed Scenarios:

- Pet CRUD Flow
- Pet Search By Status

- Place Order
- Get Order
- Create User
- Get User
- Update User
- Login User

Result:

PASS

Screenshot

```
[INFO] -----
[INFO] Running tests.PetApiTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 8.446 s -- in tests.PetApiTest
[INFO] Running tests.StoreApiTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.064 s -- in tests.StoreApiTest
[INFO] Running tests.UserApiTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.462 s -- in tests.UserApiTest
[INFO] Results:
```

12. Sample Test Output

Example:

Tests run: 6

Failures: 0

Errors: 0

Skipped: 0

BUILD SUCCESS

Screenshot

```
[INFO] Results:
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 21.243 s
[INFO] Finished at: 2026-06-25T12:58:28+05:30
[INFO] -----
PS C:\Users\299172\Documents\API_Testing>
```

13. Conclusion

Successfully developed a reusable API Automation Framework using Rest Assured and JUnit 5.

Implemented:

- CRUD Operations
- Dynamic Data Handling
- Query Parameters
- Assertions
- Reusable Specifications
- Response Model Mapping

All implemented test scenarios executed successfully.